

Omarchy Architecture Review

System design, design rationale, agent readiness and recommended improvements for an Omarchy 4.0 installation on Arch Linux

By	Sumiran Dahal
Date	28 September 2026
System reviewed	Dell Latitude 7430, Omarchy 4.0.4, stable channel
Method	Local read only audit and primary source research

Contents

1	Executive Summary
2	Scope and Method
3	System Architecture
4	Design Rationale
5	Assessment of the Agentic OS Claim
6	How an Agent Changes the System
7	Findings on This Machine
8	Recommendations
9	Conclusion
A	Sources

1. Executive Summary

Omarchy began in 2025 as an install script that cloned a git repository into the home directory. Version 4.0 turned it into a proper distribution layer. It now ships as two pacman packages, runs its own package repository and Arch mirror, and builds its own kernel. The desktop is Hyprland configured in Lua, with a single Quickshell process that replaced eight separate daemons. About 440 bash commands sit behind one `omarchy` command that documents itself.

The design follows the same thinking David Heinemeier Hansson (DHH) applied to Ruby on Rails. The author picks the tools, the parts are built to work as one system, everything is plain text, and appearance is treated as a reason people enjoy their work. Most of the less obvious parts, such as boot menu snapshots, a delayed Arch mirror and a guard against direct `pacman -Syu`, exist because Arch updates are not atomic. DHH states this himself in a planning document he later withdrew.

The claim that Omarchy is an agentic operating system is justified in one respect and incomplete in another. The system does explain itself to AI agents, and file permissions stop an agent from damaging the core. It does not yet protect the user's own configuration. Agents launch in their most permissive modes, snapshots skip the home directory, and there is no undo or record of what an agent changed.

Main recommendation. The most valuable next step is not a new language or a new base distribution. It is making each agent session reversible and limited: take a snapshot of the home directory before the session, record what changed, allow a single undo, and grant privileges per command group rather than full root for a time window.

2. Scope and Method

The review answers five questions. How is Omarchy set up on this machine? Why was each part of the stack chosen? What could be improved, ordered by effort? Does the agentic OS label hold up?

How does an AI agent actually change the system, and what did DHH put in place to make that possible?

Two workstreams ran in parallel. The first was a read only audit of the installed system. No files were changed and `sudo` was not used. Root only files such as sudoers rules were read from the cached package archive instead. The second was research using primary sources: DHH's blog, his interview on the Lex Fridman Podcast (episode 501), the omacom/omarchy repository, its release notes and the Omarchy manual.

Where a statement is my interpretation rather than something a source says, the report labels it as inferred. A few claims could not be confirmed, including DHH's reasons for choosing Chromium over Firefox and for LazyVim. Those are left out of the rationale or marked unverified.

3. System Architecture

3.1 Boot sequence

The machine boots through a chain in which each stage is managed by Omarchy's packages.

Stage	Implementation on this machine
Bootloader	Limine 12.8 with unified kernel images. <code>limine-snapper-sync</code> adds the five most recent snapshots as bootable entries.
Kernel	<code>linux-omarchy</code> 7.2.5 from the Omarchy repository. The stock Arch kernel is kept as a fallback.
Disk	LUKS2 full disk encryption, then btrfs with subvolumes for root, home, logs and the package cache.
Login	SDDM logs in automatically. The disk passphrase is the only authentication step.
Session	<code>uwsm</code> starts Hyprland 0.56 using the Lua configuration.
Desktop shell	One Quickshell process provides the bar, launcher, menus, notifications, lock screen, idle handling and the polkit password dialog.

3.2 Configuration layers

Configuration is split into four layers. This split is the most important design decision in version 4.0, because it separates what the vendor owns from what the user owns.

Layer	Location	Owner and behaviour
1. Package defaults	<code>/usr/share/omarchy</code>	Owned by root. Replaced on every update. Nobody edits it.
2. Seed files	<code>/usr/share/omarchy/config</code>	Copied into the home directory once. <code>omarchy refresh</code> copies them again after saving a backup.
3. User overrides	<code>~/.config/hypr</code> , <code>~/.config/omarchy</code>	Owned by the user. Loaded after the defaults, so they win. Reloaded on save.
4. Generated state	<code>~/.local/state/omarchy</code>	Written by Omarchy commands, for example the current theme and migration markers.

System files under `/etc`, such as sudoers rules and login manager settings, come from a second package, `omarchy-settings`. When `pacman` cannot overwrite a modified file it leaves a `.pacnew` copy for the user to merge.

3.3 Updates

The `[omarchy]` repository and the Arch mirror are both hosted by Omarchy. The mirror runs about one month behind Arch, so that broken upstream packages can be caught before users receive them. Running `omarchy update` performs the following steps in order.

1. Check free disk space and take a snapper snapshot of the root filesystem.
2. Update the keyring and run `pacman -Syu`.
3. Run pending migration scripts. There are 106, and each leaves a marker file once it has run.

4. Run user hooks, update AUR packages with yay and update developer tools with mise.
5. Remove orphaned packages and scan the logs for problems.

A pacman hook stops any system upgrade that does not come through this command. Another hook pauses Hyprland's automatic reload while pacman is writing files.

3.4 Scale

Component	Size
Command scripts	436 bash, 5 Python, about 34,000 lines
Desktop shell	103 QML files, about 35,000 lines
Routed CLI commands	367 in 63 groups, 83 require root
Base package list	147 packages
Migrations	106
Stock themes	22

3.5 Components built for AI agents

Component	What it does
Shipped skills	Two instruction sets, <code>omarchy</code> and <code>diagnose-crash</code> , are linked into the skill folders of Claude Code, Codex, OpenCode, Pi and Hermes. They tell an agent where it may write, how to back up and how to check its work. Because they are symbolic links, they update with the package.
Command catalogue	<code>omarchy commands --json</code> lists every command with its purpose, arguments and whether it needs root. The data comes from comment headers in each script.
Agent launcher	<code>omarchy agent</code> starts the chosen agent. It passes each tool's unattended flag, for example <code>claude --permission-mode auto</code> or <code>hermes --yolo</code> . This machine uses Hermes.
Crash assistant	A user service watches for crashes and offers to open the agent with the crash details and the diagnosis skill.
Privilege	Passwordless sudo is limited to three exact commands: DNS preset, browser theme colour and timezone. Other root actions go through <code>pkexec</code> , which shows a password dialog in the shell. An optional mode grants full passwordless sudo for 15 minutes.

4. Design Rationale

The table below links each major choice to the reason given in primary sources. The last column records what the choice cost, which is often where later versions spent their effort.

Choice	Stated reason	Cost
Chef's choice defaults	"It's called Omarchy because the Oma part is short for omakase... I'm the chef." DHH ties this directly to Rails.	All users receive the author's apps, including 37signals web apps and 1Password.

Choice	Stated reason	Cost
Arch instead of Ubuntu	The earlier Ubuntu project, Omakub, "kept fighting the platform instead of flowing from it."	Rolling updates can break, which led to the mirror, the guard hook and snapshots.
Hyprrland	"One of the most intoxicating ways of using a computer."	Upstream changes, such as the move from hyprrlang to Lua, forced a large migration.
Quickshell	Uses under 300 MB and is "so malleable." One themed process instead of many daemons.	Lock, idle and password prompts now depend on a single process.
Plain text everywhere	"Everything in Linux is either a config file or a CLI tool... the drawbacks... are now its major selling points."	None significant. This is the basis of the agent story.
Mandatory encryption	"Losing a laptop can't lead to a security emergency."	Autologin relies on the disk passphrase alone.
Snapshots with Limine	"Updates are not atomic, so we bolted atomicity on."	Covers the root filesystem only and requires Limine.
Own repository, mirror and kernel	To avoid AUR outages and "crash at 3 AM because of an Arch package... outside of our control."	Omarchy now carries the work of a downstream distribution.
Foot terminal	"Fast, lightweight, and compatible with even old computers."	No built in tabs. tmux fills that role.
Chromium with a small fork	Forked "just to add one feature needed for live theming."	A fork to maintain.
Bash for all commands	Inferred: no runtime, readable with <code>cat</code> , and every agent writes it well. DHH says the code is now written by agents under his direction.	Injection bugs from untrusted input. Release 4.0.2 contained eleven security fixes.

A pattern runs through these choices. DHH optimises first for how easy the system is to change and how good it looks, and adds robustness afterwards. Version 4.0 completed that work for the package and system layers. The user layer has not yet received the same treatment.

5. Assessment of the Agentic OS Claim

5.1 Where the claim holds

- 1. The system explains itself.** Shipped skills and a machine readable command list give an agent a complete map. macOS and Windows keep most settings in binary formats or graphical tools, which agents cannot use as easily.
- 2. The core is protected by permissions.** `/usr/share/omarchy` is owned by root. The skill's rule never to edit it is enforced by the file system, not only by the model's good behaviour. `omarchy refresh` always has a clean copy to restore.
- 3. Changes are text and are checked quickly.** Configuration reloads on save, and `hyprrctl configerrrors` reports problems immediately. An agent can edit, check and correct in a short loop.
- 4. There are safe places to extend.** Hooks, plugins and theme overlays let an agent add behaviour without touching core files. This machine already runs about 40 user plugins and 20 theme hooks.

5. **Root access is approved by the user.** The shell's polkit dialog lets an agent without a terminal request root access, and the user approves each request.

5.2 Where the claim falls short

Gap	Effect
Agents start in unattended mode	The agent runs as the user and can read SSH keys and browser data and change any setting without asking. The manual advises starting in plan mode, which the launcher does not do.
No rollback for user settings	Snapshots cover the root filesystem only. <code>omarchy refresh</code> restores factory files, not the user's previous state.
No change record	There is no way to list what an agent changed during a session.
Coarse privilege	The choices are a password for every request or full root for 15 minutes. Nothing in between exists, even though the command catalogue already marks which commands need root.
Rules are advice below the core	Backing up and validating depend on the agent following the skill.
Plugins are unreviewed code	Themes from git are stripped of executable files. Plugins are not, and marketplace review is still planned.

Verdict. The label is accurate for discovery and editing. For safety it is about half complete. The core is well protected, but the user's own configuration still depends on the agent behaving well.

6. How an Agent Changes the System

A simple request such as "make the window gaps smaller" passes through the following steps.

1. The agent loads the Omarchy skill from its skills folder and then reads the Hyprland guide.
2. It reads the default file in `/usr/share/omarchy/default/hypr`. Reading is allowed and writing is blocked.
3. It copies the user file to a timestamped backup, as the skill instructs.
4. It edits `~/.config/hypr/looknfeel.lua`. This file loads after the default and overrides it.
5. Hyprland reloads the change automatically. The agent runs `hyprctl configerrors` and finishes if nothing is reported.

For system changes such as installing a package, the agent runs `omarchy pkg add`. That command needs root, so it calls `pkexec`, the shell shows a password dialog and the user decides.

In summary, DHH made agent editing possible through six mechanisms: the separation of vendor and user layers, plain text that reloads on save, a command line that describes itself, skills linked into every agent, a graphical polkit agent, and snapshots before every update.

7. Findings on This Machine

The system is in good health overall. No system or user services have failed, Hyprland reports no configuration errors, no migrations are pending and all base packages are installed. The findings below are drift left behind by the upgrade to version 4.0 and by normal use.

ID	Finding	Priority	Action
F1	The SDDM configuration still points to <code>/usr/share/sddm/hyprland.conf</code> , a file no package owns. The corrected version waits as a <code>.pacnew</code> file.	High	Merge with <code>sudo pacdiff</code> .
F2	The <code>mkinitcpio</code> hooks file also has an unmerged <code>.pacnew</code> that adds a keyboard layout guard.	Medium	Merge, then rebuild the boot image.
F3	<code>omarchy debug</code> fails with "Unknown Omarchy command", although the shipped skill tells agents to run it.	Medium	Use <code>omarchy-debug --no-sudo --print</code> . Report upstream.
F4	The old 3.8.5 installation remains as a 336 MB backup folder in <code>~/.local/share</code> .	Low	Delete it.
F5	Old <code>.conf</code> Hyprland files remain beside the active Lua files. <code>hyprland.conf</code> refers to files that no longer exist.	Low	Delete the unused files.
F6	Backup files are accumulating: 15 in <code>~/.config/hypr</code> and 8 copies of <code>shell.json</code> . Ten of the Hyprland backups come from a personal plugin and are intentional.	Low	Keep the plugin history. Remove the rest.
F7	A broken autostart link for an uninstalled app logs an error at every login.	Low	Remove the link.
F8	The Plymouth theme folder has owner and permission values that differ from the package.	Low	Reinstall <code>omarchy-settings</code> .
F9	The package ships a version file that reads 4.0.0.alpha. <code>omarchy version</code> reports correctly.	Info	Report upstream.
F10	<code>~/.config</code> is not under version control, so agent changes cannot be reviewed or reverted.	High	See recommendation R3.

8. Recommendations

Recommendations are grouped into four tiers by effort and scope. Each tier builds on the previous one.

Tier 0. This machine, within one hour

ID	Recommendation
R1	Merge the pending <code>.pacnew</code> files, starting with SDDM and mkinitcpio (F1, F2).
R2	Remove the old installation, the unused Hyprland files and the broken autostart link (F4, F5, F7).
R3	Put <code>~/.config</code> under git and add a <code>post-update</code> hook that commits automatically. Add <code>etckeeper</code> for <code>/etc</code> . This provides the rollback Omarchy does not yet offer.
R4	Run agents in plan or default mode for system work. Keep auto mode for project repositories.

Tier 1. Small upstream fixes, days of effort

ID	Recommendation
R5	Let the command dispatcher find commands installed in <code>/usr/bin</code> , which fixes <code>omarchy debug</code> .
R6	Generate the version file from the package build.
R7	Show pending <code>.pacnew</code> files at the end of <code>omarchy update</code> and offer to merge them.
R8	Store backups in one state folder and keep only the latest few per file.
R9	Align the launcher with the manual so agents start in a mode that asks before acting.
R10	Publish a JSON Schema for <code>shell.json</code> and type definitions for the Lua API, so agents and editors can check changes before reload.

Tier 2. Structural changes, weeks of effort

ID	Recommendation	Detail
R11	Reversible agent sessions	Snapshot the home subvolume or commit <code>~/.config</code> when <code>omarchy agent</code> starts. Write a list of changed files when it ends. Provide <code>omarchy agent undo</code> . This is the highest value item in the report.
R12	Privilege by command group	Generate one polkit action per root command from the existing catalogue. Let the user grant groups such as package or refresh for one session instead of full root.
R13	Sandboxed agent launch	Start agents with bubblewrap or systemd sandbox options that hide SSH keys, GPG keys and browser profiles by default.
R14	MCP server for Omarchy	Generate typed tools from <code>omarchy commands --json</code> . This removes shell quoting errors and gives a natural place to log every action.
R15	Targeted bash hardening	Keep bash, but add shellcheck and tests in CI and route all untrusted input, such as notification text and video titles, through one reviewed helper. A full rewrite is not justified.
R16	Plugin trust	Pin plugins to a commit, show changes on update and declare permissions such as process, network and file access.

ID	Recommendation	Detail
R17	Lock screen fallback	Start a minimal locker automatically if the shell process crashes.

Tier 3. Base platform, months of effort

Option	Benefit	Drawback	Fit with DHH
Stay on Arch with Tier 2	Keeps speed, AUR access and current packages.	Updates stay non atomic.	High
Move to NixOS	Atomic updates and full rollback. Removes most of the workarounds.	Harder to learn. Less hackable with plain tools.	Medium. DHH drafted this plan and parked it.
Image based system	Atomic updates and simple testing.	Users lose local package freedom.	Low. Rejected in DHH's plan.
CachyOS base	Faster kernels and packages.	Still has Arch's update model.	Low

Tier 2 on Arch should come first. Agents mostly edit the user layer, and that layer remains ordinary files even on NixOS, so a new base would not solve the main safety gap. A move to Nix becomes worthwhile only when maintaining the Arch workarounds costs more than the migration.

9. Conclusion

Omarchy 4.0 is a coherent system with a clear philosophy. Its separation of vendor files from user files, its self describing commands and its shipped agent skills make it one of the easiest desktops for an AI agent to operate. The same openness means an agent can change anything the user owns, and the tools to review or reverse those changes are not yet in place.

For this machine, merging the pending configuration files and putting `~/.config` under version control will resolve the current drift and provide a safety net. For the project, reversible and limited agent sessions would do more for the agentic OS goal than any change of language or base distribution.

Appendix A. Sources

Source	Location
Omarchy repository and release notes	github.com/omacom/omarchy
Omarchy manual	omarchy.org/manual and learn.omacom.io
Lex Fridman Podcast, episode 501	lexfridman.com/dhh-2-transcript
DHH, Omarchy 2.0	world.hey.com/dhh/omarchy-2-0-16fefc15
DHH, The malleable computer	world.hey.com/dhh/the-malleable-computer-7c187a9b
DHH, Promoting AI agents	world.hey.com/dhh/promoting-ai-agents-3ee04945
Withdrawn Nix plan	omacom/omarchy commits 97a86af1 and f38eb420
Hyprland Lua announcement	hyprland/news/26_lua
Docker group escalation report	0xcc.io/posts/omarchy-root-creds
Local audit and full research notes	omarchy-research-architecture/evidence